

My Yarn Stash

Photo of the yarn label, AI auto-fills everything, matches to Ravelry patterns from your stash. Plus shop commissions.

What it is

My Yarn Stash is an iOS + Android app for hobby knitters and crocheters. Photograph the yarn label, AI extracts material, yardage, needle recommendation, manufacturer and colour. The app then matches Ravelry patterns the user can actually knit with their current stash. Plus: project tracking, FO logbook (finished objects), pattern-match history, and group sharing for knit circles. Two revenue rails: 50% on Premium subs AND a share of Awin shop commissions when the referred user buys yarn in-app.

Who it is for

Hobby knitters, crocheters and fibre artists from beginners with a single shelf to power users with several hundred labels. Audience skews ~90% female, high purchasing power (yarn re-bought every 3-6 weeks), EU + US + UK + AU. Creator archetypes: independent pattern designers, KnittingTok and CrochetTok creators, indie yarn dyers, stash-tour reel makers, finished-project carousel creators, granny-square content, fibre-festival recap creators.

Key features

- Vision-AI label scan: material, yardage, needle, manufacturer, colourway, weight from one photo.
- Pattern matching: stash items matched against Ravelry patterns. "Patterns I can knit right now from my stash".
- Project tracking + FO logbook: in-progress with photo, finished objects archive, yardage used per project.
- Group sharing for knit circles: 1 small group on Free (5 members), unlimited on Premium.
- Awin in-app shop rail: deep-linked Knit Picks + Minerva, no separate browser flow.
- Discover tab for pattern designers.

Business model

- Two revenue streams: recurring Premium subscription + Awin shop commission share.
- Premium pricing: 4.99 USD / month or 29.99 USD / year (no annual discount, no free trial).
- Free tier generous: 30 yarn items, 1 group up to 5 members, manual + photo scan, pattern-match search.
- Premium unlocks: unlimited stash, unlimited groups, AI pattern-match, advanced tracking.
- Free Lifetime Premium for onboarded affiliates.

Pricing

- Premium: 4.99 USD / month or 29.99 USD / year.
- Shop rail: Klar earns ~5-10% Awin commission per basket; affiliate share is a portion for 24 months from referred user's first purchase.
- ASC live since May 2026; affiliate share is always 50% of the then-current Premium price.

Market gap - what was missing

Ravelry's mobile experience has been broken since the 2020 redesign disaster. Ravelry has 10M+ registered users, most maintain stash and pattern queues, but the official mobile app is functionally abandoned - accessibility complaints led to the redesign rollback, the underlying tech is still 2015-era, and there is no AI-anything. Every knitter has a phone, a physical stash growing in their living room, and the chronic "I know I have yarn for this pattern, I just can't remember" pain. That is the gap. My Yarn Stash fills it with better UX, AI auto-fill (label scan nothing else offers), and pattern-match against the user's actual stash. The Awin shop rail is the second unlock - knitters re-buy yarn constantly, so a single converted user's lifetime affiliate value compounds on every restock.

Why now

Three shifts make My Yarn Stash possible in 2026. First, Vision LLM costs dropped enough that scanning every yarn label at 0.005-0.01 USD per call is economically viable inside a 5 USD/month price point. Second, Ravelry's API access has been stable enough since 2024 that pattern matching against a live stash is reliable. Third, Awin's iOS-deep-link infrastructure matured enough in 2024-2025 that in-app yarn purchases attribute cleanly through to the affiliate.

Affiliate compensation

- 50% of net Premium subscription revenue for 24 months from each attributed user's first paid month.
- 30-day refund holdback before payout becomes claimable (Apple / Google refund window).
- 50 EUR minimum payout, monthly, via Wise.
- Free Lifetime Premium for onboarded affiliates (no posting obligation).
- Per-handle dashboard with live click / install / sub / earning split, no agency layer.

How tracking works - DIY attribution stack

Most affiliate stacks ride on top of Branch.io, AppsFlyer or a similar paid attribution SDK. We built our own. Those tools charge per attributed install, they add a third party to the user's first-launch experience, and they make influencer payouts opaque. We wanted full control of every step from click to Wise transfer. The full path has six stages - link click, install, first launch attribution, Premium purchase, holdback, payout. Here is what runs in each stage.

Why we built our own instead of Branch.io

Three reasons. (1) Cost: Branch.io and AppsFlyer charge 0.20-0.40 USD per attributed install at our volume tier, which would have absorbed roughly 8-12% of the affiliate payout pool. With DIY we keep that for affiliates. (2) Privacy: each additional SDK reads first-launch context - that is another vendor with first-touch user data we did not want to add. Apple's App Tracking Transparency framework has also made probabilistic fingerprinting unreliable since iOS 14.5; the paid SDKs work around this with opaque heuristics that we could not audit. (3) Transparency: when a dashboard number looks wrong, we read our own code in 10 minutes instead of debugging a third-party black box. The trade-off is that we own the bugs. Every line of attribution code is in our repo.

Android: Google Play install-referrer

Android gives us the cleanest path. When a user taps your link, the Google Play Store install carries the original referrer string (a URL-encoded query parameter from the redirect chain) through to first launch via the Play Install Referrer API. On first launch the app reads `getInstallReferrer().getInstallReferrer()`, we parse the handle and source out of it, and write an attribution row in our Supabase table alongside the install event. Deterministic. No fingerprinting. No clipboard.

iOS warm (app already installed)

If the user already has My Yarn Stash installed, iOS routes the tap through a Universal Link directly into the app with the handle in the URL path. We register `yarnstash.app/i/*` as a Universal Link in the iOS entitlements file, Apple's CDN serves the `apple-app-site-association` JSON we publish at the domain root, and the OS routes the tap. Inside the app, the deep-link handler reads the URL path, extracts the handle, and writes the attribution row. Deterministic.

iOS cold (~99% of creator traffic)

The hard case - and ~99% of creator traffic - is iOS cold install. iOS gives the freshly installed app no context about how it was reached. Branch.io's whole business is solving this with device-fingerprinting heuristics that became increasingly unreliable after iOS 14.5. We do it differently. When a user taps your link in Safari, our redirect chain hits `yarnstash.app/i/[handle]`. That page writes a short-lived JSON token to the clipboard before redirecting to the App Store. The token contains: handle, timestamp (issued-at), app-key, version, and an HMAC-SHA256 signature computed against a server-side secret. After install, on first launch, the app reads the clipboard once (Apple shows the standard clipboard-paste system UI), parses the JSON, verifies the signature against the public verification logic, accepts the token if it is freshly signed and less than 7 days old, and writes the attribution row. The user sees no extra screen when it works.

Optional first-open confirmer

Optional fallback. On first open, after we have read whatever the clipboard provided, the app shows a single confirmation line: "Invited by @[suggested-handle]? Yes / No". The handle field is pre-filled with whatever the clipboard suggested; if the clipboard was empty, the field is empty and the user can type. Most users tap Yes. This catches the edge case where Apple's clipboard-access dialog was denied, the user copied something else between tap and install, or the clipboard expired. The step is toggleable per app.

RevenueCat webhook -> payout queue

Premium purchases flow through RevenueCat. RC handles Apple StoreKit and Google Play Billing receipts, dedup, refund detection, and converts everything into a consistent webhook event format. When a `INITIAL_PURCHASE`, `RENEWAL`, `PRODUCT_CHANGE`, or `CANCELLATION` event fires, RC posts JSON to our Supabase Edge function `affiliate-rc-webhook`. The function (1) verifies the RC signature header against our shared secret to confirm authenticity, (2) idempotency-checks the event ID against the `processed_webhook_events` table to avoid double-credit on RC retries, (3) looks up the user's attribution row, (4) computes 50% of the net revenue after Apple's or Google's 15-30% cut, (5) credits the `affiliate_earnings` table with handle, app, gross, cut, net, our-share, holdback-until date, and the source event ID. All in one Postgres transaction.

Holdback -> batch -> Wise

Each credited earning enters a 30-day refund holdback (the Apple/Google refund window). After holdback expires the row flips to `claimable`. A `pg_cron` job runs `build_payout_batch()` on the 5th of each month: it groups all `claimable` rows per affiliate handle, applies the 50 EUR minimum-payout threshold (handles

Why this works as an affiliate pitch

Ravelry's mobile app is genuinely dead since the 2020 redesign disaster - 10M+ users without a native solution. My Yarn Stash fills the gap with better UX and AI auto-fill that nothing else offers. The pattern-match feature solves "stash blindness". Two revenue rails mean affiliates earn on subs AND on every yarn purchase by referred users - knitters re-buy yarn every 3-6 weeks, so the Awin tail compounds meaningfully over 12-24 months.

On posts with affiliate link: #ad / #sponsored / Paid Partnership (FTC, EU disclosure law).